

TryHackMe — Computer Fundamentals Module

Linux CLI Basics

Learning Notes — Navigation, File Ops, System Info, Pipes & Permissions

"Most of our work happens in the Linux terminal. Get comfortable here and you'll be ready for anything in cyber security." — Your Supervisor, Mission Brief

 Navigation	 File Operations	 System Info	 Pipes & Filters	 Permissions	 Lab Mission
--	---	---	---	---	---

1. Introduction — Why the CLI Matters

The Linux Command-Line Interface (CLI) is the primary tool for navigating servers, running security tools, and setting up environments in cyber security. In this room you play a new IT Support Engineer whose supervisor left in a hurry — leaving you a note to find the mission_brief.txt file and gather a system report. Every task is solved using the Linux terminal.

CLI stands for...

Command-Line Interface — a text-based interface for controlling a computer by typing commands instead of clicking a graphical interface. Room answer: What does 'CLI' stand for? → Command-Line Interface

1.1 Why CLI Over GUI for Security Work?

Factor	GUI	CLI
Speed	Multiple clicks required for each action	Single command executes instantly
Automation	Repetitive tasks require repeated clicking	Scripts automate any sequence of commands
Remote access	Requires RDP/VNC — high bandwidth	SSH works even on slow connections



pwd — Print Working Directory — where am I?

Syntax: `pwd`

Common flags:

- `-L` Use logical path (default) — follows symlinks
- `-P` Use physical path — resolves symlinks to real location

Examples:

- `pwd` # shows current directory, e.g. `/home/ubuntu`
- `ubuntu@tryhackme:~$ pwd`
- `/home/ubuntu`

Security use: Attackers and defenders alike use `pwd` constantly to confirm their location in the file system. In a script, `pwd` verifies you are in the correct directory before performing destructive operations.



ls — List — what files and folders are here?

Syntax: `ls [options] [directory]`

Common flags:

- `-l` Long format — shows permissions, owner, size, date (use this constantly)
- `-a` Show all files including hidden (starting with `.`)
- `-la` Combine both — long format including hidden files
- `-h` Human-readable file sizes (KB, MB, GB) — use with `-l`
- `-t` Sort by modification time (newest first)
- `-r` Reverse sort order
- `-R` Recursive — list all subdirectories

Examples:

- `ls` # list current directory
- `ls -l` # detailed listing with permissions and sizes
- `ls -la` # all files including hidden (dotfiles)
- `ls -lah /etc` # list /etc with human-readable sizes
- `ls -lt` # sort by modification time

Security use: `ls -la` reveals hidden files (dotfiles like `.bash_history`, `.ssh`) that contain sensitive data. Security professionals check `ls -la` as a first step when investigating a compromised account. Malware often hides as dotfiles.



cd — Change Directory — move between folders

Syntax: `cd [directory]`

Common flags:

- `cd ~` Go to home directory (same as just typing `cd`)
- `cd ..` Go up one level (parent directory)

- `cd ../..` Go up two levels
- `cd /` Go to root directory (top of file system)
- `cd -` Return to previous directory
- `cd /full/path` Navigate using absolute path

Examples:

- `cd Documents` # move into Documents (relative)
- `cd /home/ubuntu` # move using absolute path
- `cd ..` # go up one level
- `cd ~` # return to home directory
- `cd -` # go back to previous directory

Security use: Understanding absolute vs relative paths is critical for scripting and navigation. Penetration testers navigate file systems constantly to find credentials, config files, and sensitive data. `cd /etc → cat passwd/shadow` is a classic sequence.

3. Finding & Reading Files

Once you can navigate, the next skill is finding files anywhere on the system and reading their contents. `find` is the powerful search utility; `cat` is the simplest way to read a file's contents.



find — Search for files anywhere on the file system

Syntax: `find [starting_point] [options] [expression]`

Common flags:

- `-name filename` Search by exact name (case-sensitive)
- `-iname filename` Search by name (case-insensitive)
- `-type f` Find files only (not directories)
- `-type d` Find directories only
- `-user username` Find files owned by a specific user
- `-size +10M` Find files larger than 10MB
- `-mtime -7` Find files modified in last 7 days
- `-perm /4000` Find SUID files (privilege escalation target)

Examples:

- `find ~ -name mission_brief.txt` # find file in home dir
- `find / -name '*.conf' 2>/dev/null` # find all config files
- `find /home -type f -name '*.txt'` # find text files in /home
- `find / -perm /4000 2>/dev/null` # find SUID binaries

Security use: `find / -perm /4000` is one of the first commands run during Linux privilege escalation — it lists all SUID binaries, which can be exploited to gain root. `find / -name id_rsa` locates SSH private keys. `2>/dev/null` suppresses permission errors.

**cat** — Concatenate — read and display file contents**Syntax:** `cat [options] [file]`**Common flags:**

- `-n` Show line numbers
- `-A` Show all special characters (tabs, line endings)
- `-b` Number non-blank lines only

Examples:

- `cat mission_brief.txt` # read a file
- `cat /etc/passwd` # view user account list
- `cat /etc/os-release` # view Linux distribution info
- `cat -n script.sh` # view with line numbers
- `cat file1 file2` # concatenate two files

Security use: `cat /etc/passwd` reveals usernames and shell assignments. `cat /etc/os-release` reveals the exact Linux distribution. Attackers use `cat` to read configuration files containing credentials, API keys, and database connection strings.

4. File & Directory Operations

Beyond reading files, the CLI provides powerful commands for creating, copying, moving, and deleting files and directories. These are the fundamental file management operations — the Linux equivalent of right-click menus.

Command	Syntax	What It Does
<code>mkdir</code>	<code>mkdir [dir]</code>	Create a new directory. <code>-p</code> flag creates parent directories as needed: <code>mkdir -p a/b/c</code>
<code>rmdir</code>	<code>rmdir [dir]</code>	Remove an empty directory. Fails if the directory contains files. Use <code>rm -r</code> for non-empty.
<code>touch</code>	<code>touch [file]</code>	Create an empty file, or update a file's timestamp if it already exists.
<code>cp</code>	<code>cp [src] [dest]</code>	Copy a file or directory. <code>-r</code> flag copies directories recursively: <code>cp -r dir1 dir2</code>
<code>mv</code>	<code>mv [src] [dest]</code>	Move or rename a file/directory. No <code>-r</code> needed — <code>mv</code> works on directories directly.
<code>rm</code>	<code>rm [file]</code>	Delete a file. <code>-r</code> deletes directories recursively. <code>-f</code> forces deletion without confirmation. WARNING: irreversible!
<code>nano</code>	<code>nano [file]</code>	Simple text editor built into most Linux systems. <code>Ctrl+O</code> saves, <code>Ctrl+X</code> exits.

echo	echo [text]	Print text to screen. echo 'text' > file writes to a file. >> appends.
------	-------------	--

```
# Create a directory structure
mkdir -p /tmp/investigation/logs

# Copy a file
cp mission_brief.txt /tmp/backup_mission.txt

# Move/rename a file
mv old_name.txt new_name.txt

# Create a file and write to it
echo 'Hello Linux' > hello.txt
echo 'Second line' >> hello.txt    # append

# DANGER — this permanently deletes the directory and all contents:
rm -rf /tmp/test_folder    # use with extreme caution!
```

rm -rf Warning

rm -rf / would delete the entire file system — and Linux will let you do it if you have root permissions. Always double-check rm commands before pressing Enter. There is no Recycle Bin in the CLI. rm -rf is permanent and instant. Use rm -i for interactive mode that asks before deleting each file.

5. System Information Commands

After navigating to the mission_brief.txt file, it tasks you with gathering a system report: who you are, the kernel version, total disk space, and the Linux distribution name. These system information commands are essential for understanding any Linux environment.

Command	What It Shows	Key Flags & Details
whoami	Your current username	No flags needed. Essential first command when you gain access to a system. Shows which user account you are operating as.
uname -a	Full system information	-a gives all info: kernel name, hostname, kernel version, build date, machine type, OS. Most useful form. Just 'uname' gives only 'Linux'.

<code>df -h</code>	Disk space usage	-h = human-readable sizes (GB/MB). Shows each mounted filesystem, total/used/available space, and mount point.
<code>du -sh</code>	Directory size	-s = summary (total only), -h = human-readable. e.g. <code>du -sh /home/ubuntu</code> shows total size of home directory.
<code>free -h</code>	RAM usage	-h = human-readable. Shows total, used, free, shared, cache, available memory. Swap space also shown.
<code>cat /etc/os-release</code>	Linux distribution name	Shows the distribution name (Ubuntu, Debian, Kali), version, and codename. Essential for identifying the OS type.
<code>lsb_release -a</code>	Distribution details	Shows distributor ID, description, release number, and codename. Part of the lsb-release package.
<code>id</code>	User ID and group memberships	Shows UID, GID, and all group memberships. Critical for privilege checking — is the user in the sudo or docker group?
<code>hostname</code>	Machine's hostname	The computer's network name. -I flag shows all IP addresses assigned to the machine.
<code>uptime</code>	System uptime and load	Shows how long the system has been running, number of users, and CPU load averages.

```
# Who am I?
ubuntu@tryhackme:~$ whoami
ubuntu

# Full system info — kernel, hostname, architecture
ubuntu@tryhackme:~$ uname -a
Linux tryhackme 5.15.0-1064-aws #17-Ubuntu SMP x86_64 GNU/Linux

# Disk space (human-readable)
ubuntu@tryhackme:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/xvda1      7.7G  4.2G  3.5G   55% /

# Linux distribution name
ubuntu@tryhackme:~$ cat /etc/os-release
NAME='Ubuntu'
VERSION='20.04.6 LTS (Focal Fossa)'
ID=ubuntu
```

6. Pipes, Redirection & Filters

One of Linux's most powerful features is the ability to chain commands together using pipes (|) and redirect output to files. This allows complex operations to be built from simple commands — and is fundamental to both system administration and security work.

6.1 Pipes — Chaining Commands

The pipe character | takes the output of one command and feeds it as input to the next. This lets you build powerful command pipelines:

```
# Count how many files are in a directory
ls -la | wc -l

# Filter process list for a specific process
ps aux | grep apache

# View long output one page at a time
cat /etc/passwd | less

# Find all .txt files and search them for the word 'password'
find /home -name '*.txt' | xargs grep -l 'password'

# Count occurrences of a word in a file
cat access.log | grep '404' | wc -l
```

6.2 Redirection — Writing Output to Files

Operator	What It Does
>	Redirect stdout to a file — OVERWRITES the file if it exists. e.g. ls > filelist.txt
>>	Redirect stdout to a file — APPENDS to the file. e.g. echo 'new entry' >> log.txt
<	Redirect stdin from a file — feed a file as input to a command. e.g. sort < unsorted.txt
2>	Redirect stderr (error messages) to a file. e.g. find / -name '*.conf' 2> /dev/null
2>&1	Redirect both stdout and stderr to the same destination.
	Pipe — send stdout of left command as stdin to right command. e.g. ls -la grep '.txt'

6.3 Essential Filter Commands

Command	Function	Example
grep	Search text for a pattern	grep 'password' config.txt grep -r 'root' /etc/
sort	Sort lines alphabetically/numerically	sort names.txt ls -l sort -k5 -n (sort by size)
uniq	Remove duplicate consecutive lines	sort access.log uniq -c (count occurrences)
wc	Count words, lines, characters	wc -l file.txt (line count) ls wc -l (count files)
head	Show first N lines (default 10)	head -5 log.txt head -1 /etc/passwd
tail	Show last N lines (default 10)	tail -f /var/log/syslog (follow live) tail -20 log.txt
cut	Extract columns from text	cut -d: -f1 /etc/passwd (extract usernames)
awk	Pattern scanning and processing	awk '{print \$1}' file.txt (print first column)
sed	Stream editor — find and replace	sed 's/old/new/g' file.txt

7. File Permissions

Linux file permissions control who can read, write, or execute each file and directory. Understanding permissions is essential for both using Linux correctly and for identifying security misconfigurations during penetration tests.

7.1 Reading Permission Strings

# Output of ls -la:									
-rwxr-xr--	1	ubuntu	staff	4096	Jan 15 09:30	script.sh			
# ^	^	^	^	^	^	^	^		
#								filename	
#								file size (bytes)	
#								last modified	
#								group owner	
#								user owner	
#								hard link count	

```

# | | | other permissions (r--)
# | | group permissions (r-x)
# | user permissions (rwx)
# (file type: - file, d
dir, 1 link)

```

7.2 Permission Characters

Symbol	Permission	Value (Octal)	Meaning on File / Directory
r	Read	4	File: view contents. Directory: list contents (ls).
w	Write	2	File: modify contents. Directory: create/delete files inside it.
x	Execute	1	File: run as program. Directory: enter the directory (cd).
-	No permission	0	That permission is denied for that category of user.

7.3 The Three Permission Groups

Group	Who It Applies To	Position in Permission String
User (u)	The file's owner — the user who created it or was assigned ownership.	Characters 2–4 of the permission string (rwx_____)
Group (g)	Users who belong to the file's assigned group.	Characters 5–7 of the permission string (____rwx__)
Other (o)	Everyone else on the system — not the owner, not in the group.	Characters 8–10 of the permission string (_____rwx)

7.4 Changing Permissions — chmod

```

# chmod — change file permissions

# Symbolic method:
chmod u+x script.sh    # add execute for user
chmod g-w file.txt     # remove write for group
chmod o+r file.txt     # add read for others

```

```
chmod a+x script.sh      # add execute for all (a = all)

# Octal method (most common in practice):
chmod 755 script.sh      # rwxr-xr-x (user=7, group=5, other=5)
chmod 644 config.txt     # rw-r--r-- (user=6, group=4, other=4)
chmod 600 id_rsa         # rw----- (user only - correct for SSH keys!)
chmod 777 file.txt       # DANGEROUS - everyone can read/write/execute

# chown - change file owner
chown ubuntu:staff file.txt # change owner and group
chown root file.txt        # change owner to root
```

Common Permission Values

755 (rwxr-xr-x) — standard for scripts and directories. 644 (rw-r--r--) — standard for regular files. 600 (rw-----) — SSH private keys MUST have this. 777 (rwxrwxrwx) — NEVER use on sensitive files. 4755 — SUID bit set — runs as file owner regardless of who executes it — major privilege escalation risk.

8. Additional Useful Commands

Command	Purpose	Key Usage
man	Manual pages	man ls — full documentation for any command. Press q to quit. Space for next page. /search to search.
history	Command history	Shows all previously run commands. !! reruns last command. !n reruns command number n. Ctrl+R reverse searches.
clear	Clear the screen	Clears all text from the terminal window. Ctrl+L does the same thing faster.
sudo	Run as root	sudo command — run as superuser. sudo su — become root. sudo -l — list what you can run as sudo.
su	Switch user	su username — switch to another user. su - — switch to root (with environment).
ps	Process list	ps aux — list all running processes with details. Used to find running malware.
kill	Terminate process	kill PID — graceful stop. kill -9 PID — force kill. killall name — kill all processes by name.
top/htop	Live process monitor	top — built-in. htop — improved version (may need install). Real-time CPU/RAM and process list.

wget	Download files	wget URL — download a file from the internet. -O filename — save with a different name.
curl	Transfer data	curl URL — fetch URL content. -o save to file. -I show headers only. Used heavily in security testing.
ssh	Secure remote shell	ssh user@host — connect to remote Linux server. The primary way to manage remote Linux systems.
netstat	Network connections	netstat -tulpn — show all listening ports and which process is using them. ss -tulpn is the modern replacement.
ip addr	Network interfaces	ip addr — show all network interfaces and IP addresses. ifconfig is the legacy command.
env	Environment variables	env — list all environment variables. echo \$PATH — show the PATH variable. Export to set.

9. The Lab Mission — Step by Step

The room is structured as a practical mission. You play a new IT Support Engineer whose supervisor left a note. Your job: find the mission_brief.txt file, read it, then gather the requested system information. Here is the complete walkthrough:

9.1 Part 1 — Navigation Mission: Find the Missing Notes

Step	Command	What You Learn
1	pwd	Confirm you start in /home/ubuntu — your home directory.
2	ls	List the home directory: Desktop, Downloads, Documents, logs, projects, etc.
3	ls -la	Add -la to see hidden files (dotfiles starting with .) alongside normal files.
4	find ~ -name mission_brief.txt	Search the entire home directory for the file. Linux prints the full path when found.
5	cd /path/to/folder	Navigate to the directory containing mission_brief.txt using the path find returned.
6	ls	Confirm mission_brief.txt is present in the current directory.
7	cat mission_brief.txt	Read the file — it contains the FLAG and your next assignment (system report).

9.2 Part 2 — Investigating the System

After reading mission_brief.txt, your next assignment is to gather a system report. The file requests four pieces of information:

Required Info	Command	What to Note
Who you're logged in as	<code>whoami</code>	Returns your username (ubuntu). Confirms your identity on the system.
The kernel version	<code>uname -a</code>	Returns full system info including kernel version string.
Total disk space	<code>df -h</code>	Returns disk usage for all filesystems. Note the total size and % used.
Linux distribution name	<code>cat /etc/os-release</code>	Returns NAME='Ubuntu' — the distribution identifier.

9.3 Part 3 — Mini Challenge

The final task is a mini challenge where you apply everything you have learned to complete a task independently. The typical challenge involves:

- Using `find` to locate a specific file hidden somewhere on the system.
- Navigating to it with `cd` and reading it with `cat`.
- Extracting a flag value from the file contents.

```
# Typical mini challenge workflow:
find / -name '*.txt' 2>/dev/null | grep -v proc
# Or search by owner:
find / -user ubuntu -type f 2>/dev/null
# Or search in common locations:
find /home /var /tmp /opt -name '*.txt' 2>/dev/null
# Read any interesting findings:
cat /path/to/found/file.txt
```

10. Room Answers — Quick Reference

Question	Answer
What does 'CLI' stand for?	Command-Line Interface
What command prints your current working directory?	<code>pwd</code>
What flag shows hidden files with <code>ls</code> ?	<code>-a</code>

What command shows all files in long format including hidden?	<code>ls -la</code>
What command is used to locate a specific file?	<code>find</code>
What command reads the content of a file?	<code>cat</code>
What command shows who you are logged in as?	<code>whoami</code>
What command shows the kernel version and system info?	<code>uname -a</code>
What command shows disk space in human-readable format?	<code>df -h</code>
What file contains the Linux distribution name?	<code>/etc/os-release</code>
What symbol represents the home directory?	<code>~</code>
What command goes up one directory level?	<code>cd ..</code>

11. Security Implications

The Terminal Is the Attacker's Primary Interface

Once an attacker gains shell access to a Linux system, every technique they use from that point involves the CLI. They navigate with `cd`/`ls`, find sensitive files with `find`, read credentials with `cat`, check privileges with `id`/`sudo -l`, and escalate with `chmod`/SUID exploits. Knowing the CLI from both sides — attacker and defender — is fundamental to security work.

`find / -perm /4000` — The Privilege Escalation Command

SUID binaries (set-uid bit) run with the permissions of the file owner regardless of who executes them. If root owns a SUID binary, anyone who runs it gains temporary root privileges. `find / -perm /4000 2>/dev/null` lists all SUID binaries on the system — the first thing a penetration tester runs after gaining a shell. GTFOBins (gtfobins.github.io) documents how to exploit them.

`cat /etc/shadow` — The Password Hash File

On Linux, password hashes are stored in `/etc/shadow` (readable only by root). If an attacker gains read access to `/etc/shadow` (via misconfigured permissions or a root shell), they can extract all password hashes for offline cracking with tools like `hashcat` or `john`. Regular auditing: `ls -la /etc/shadow` should always show `-----` (not world-readable).

History Reveals Everything

The command history (`cat ~/.bash_history`) records every command a user has run. This is invaluable for forensic investigation — you can reconstruct exactly what happened on a compromised system. It is also a risk: credentials entered on the command line (e.g. `mysql -u root -pSECRETPASSWORD`) are permanently saved to history. Always unset `HISTFILE` or use password managers instead.

World-Writable Files Are a Privilege Escalation Vector

`find / -writable -type f 2>/dev/null` finds all world-writable files. If a root-owned script or cron job file is world-writable, an attacker can modify it to execute arbitrary commands as root on the next execution. This is one of the most common privilege escalation paths on improperly configured Linux systems.

12. Key Takeaways & Command Cheat Sheet

12.1 Key Takeaways

- CLI = Command-Line Interface. The primary tool for Linux — faster, more powerful, and more automatable than GUI.
- Terminal prompt: `ubuntu@tryhackme:~$` — username, hostname, current directory (~=home), privilege level (\$=user, #=root).
- The three navigation commands: `pwd` (where am I?), `ls` (what is here?), `cd` (move me there).
- `ls -la` shows all files including hidden (dotfiles). Hidden files start with a dot (.).
- `find ~ -name filename` — find a file in the home directory. `find /` searches the whole system.
- `cat` reads file contents. `less` is also useful for paging through long files (q to quit).
- `whoami`, `uname -a`, `df -h`, `cat /etc/os-release` — the system report commands from the mission brief.
- Pipes (|) chain commands. `>` overwrites file. `>>` appends. `2>/dev/null` hides errors.
- Permission string: `-rwxr-xr--` = type + user perms + group perms + other perms.
- `chmod 755 = rwxr-xr-x` (execute). `chmod 600 = rw-----` (SSH key). `chmod 777 = DANGEROUS`.
- `rm -rf` is permanent — no recycle bin. Always double-check before pressing Enter.

12.2 Essential Command Cheat Sheet

Command	What It Does
<code>pwd</code>	Print Working Directory — shows where you currently are.
<code>ls</code>	List directory contents. <code>-l</code> detail. <code>-a</code> all/hidden. <code>-la</code> both. <code>-h</code> human sizes.
<code>cd [dir]</code>	Change Directory. <code>cd ~</code> home. <code>cd ..</code> parent. <code>cd -</code> previous. <code>cd /</code> root.
<code>find [path] -name [file]</code>	Search for files by name. <code>find ~ -name file.txt</code> <code>find / -perm /4000</code>
<code>cat [file]</code>	Display file contents. <code>-n</code> line numbers. <code>cat /etc/os-release</code> for distro info.
<code>mkdir [dir]</code>	Create directory. <code>-p</code> creates parent directories.

<code>rm [file]</code>	Delete file. -r recursive (dirs). -f force. WARNING: permanent!
<code>cp [src] [dest]</code>	Copy file. -r for directories.
<code>mv [src] [dest]</code>	Move or rename file/directory.
<code>echo [text]</code>	Print text. > write to file. >> append to file.
<code>whoami</code>	Show current username.
<code>uname -a</code>	Show kernel version and system info.
<code>df -h</code>	Show disk space usage (human readable).
<code>id</code>	Show user ID and group memberships.
<code>ps aux</code>	List all running processes.
<code>grep [pattern] [file]</code>	Search for pattern in file. -r recursive. -i case-insensitive. -l list files only.
<code>sudo [command]</code>	Run command as root. sudo su to become root.
<code>chmod [perms] [file]</code>	Change file permissions. 755 scripts. 644 files. 600 SSH keys.
<code>chown [user] [file]</code>	Change file owner.
<code>history</code>	Show command history. Ctrl+R reverse search.
<code>man [command]</code>	Show manual page for any command.

12.3 Next Steps on TryHackMe

- **Immediate:** Continue with the Linux Fundamentals 1, 2 & 3 rooms — deeper CLI navigation, user management, and processes.
- **Security:** Complete the Linux Privilege Escalation room — apply find, cat, and chmod knowledge offensively.
- **Scripting:** Study the Bash Scripting room — automate everything you learned in this room with scripts.
- **Applied:** Take the Grep room — master pattern matching for log analysis and security investigations.
- **IR:** Work through the Linux Incident Surface room — use CLI commands for live incident response.

The Terminal is Home

Every professional cyber security skill — from running Nmap to analysing logs to deploying exploit code — happens at the terminal. Every room, every challenge, and every real-world engagement you encounter will require you to be fluent in Linux CLI. The commands

learned in this room (pwd, ls, cd, find, cat, grep, chmod) are used every single day by professionals at every skill level. Practise them until they are instinctive.

Source: TryHackMe — Linux CLI Basics Room | tryhackme.com/room/linuxclibasics